

SIH 2026 — Shortlisted Problem Statement Deep-Dive

Three candidate problem statements, evaluated for the internal college round. Each section covers: the problem as stated, our proposed solution, what already exists and how we differ from it, advantages, disadvantages, and the single hardest part of building it — with a concrete plan to get past that.

1. SIH26183 — Real-Time Identification of Fraud-Linked Cryptocurrency Exchanges from Victim-Reported Suspect Wallet Addresses

Theme: Blockchain & Cybersecurity · **Sponsoring Ministry:** Ministry of Home Affairs

Problem Statement

When a citizen reports a crypto scam (e.g., through the 1930 cybercrime helpline or cybercrime.gov.in), investigators are handed one artifact: a suspect wallet address. From there, manually tracing where the stolen funds actually went — which exchange they were cashed out through — is slow, manual, and requires specialist blockchain-forensics skill that most cybercrime cells don't have in-house. The PS asks for an automated system that takes a victim-reported address and identifies, in real time, the fraud-linked exchange(s) the funds moved through.

How We're Going to Solve It

1. **Ingest** the reported wallet address and pull its full public transaction history via free blockchain APIs (Blockchair, Blockchain.com, Etherscan for the EVM side).
2. **Graph construction** — build a directed transaction graph outward from the seed address, multiple hops deep, weighted by amount and time-decay (money that moved 10 minutes ago matters more than money that moved 2 years ago).
3. **Entity clustering** — apply the standard heuristics real investigators use: common-input-ownership clustering (addresses spent together in one transaction usually belong to one wallet), and change-address detection, to collapse hundreds of raw addresses into a handful of probable "actor clusters."

4. **Obfuscation detection (our novel layer)** — train a lightweight classifier/anomaly scorer on transaction *shape* (rapid fan-out into many addresses followed by fan-in, round-number splitting, timing bursts) to flag likely mixing/layering hops, rather than presenting a flat graph and asking a human to spot the pattern.
5. **Exchange attribution** — match cluster exit points against open attribution data (labeled deposit-address datasets such as WalletExplorer, GraphSense TagPacks, and public sanctions lists like OFAC's SDN crypto addresses) to say "funds most likely exited via Exchange X, deposit cluster Y."
6. **Output** — a one-page, auto-generated investigator report: a visual fund-flow graph, a confidence-scored "most likely cash-out point," and the evidence trail behind that score, exportable as a PDF that can go straight into a case file.

Current Solutions & How Ours Differs

Existing tool	What it does	Gap for our use case
Chainalysis Reactor / KYT	Industry-standard graph tracing, entity clustering, real-time risk scoring — used by 150+ public-sector orgs globally	Enterprise licensing (lakhs of dollars/year), not accessible to a district cybercrime cell or a student-built pilot
TRM Labs, Elliptic Investigator	Similar clustering + attribution, TRM notably uses probabilistic entity-tracking across chain-hops and swaps	Same licensing barrier; closed-source, black-box scoring
Crystal Blockchain, Merkle Science	AI-driven clustering, law-enforcement-workflow focused	Merkle Science is one of the few actually built for Indian law enforcement, but still a paid enterprise product, not something a local cell can self-host or customize
Etherscan, Blockchair, Arkham, MetaSleuth (OSINT tools)	Free, public, good for manual one-hop lookups	No automated clustering, no obfuscation detection, no confidence scoring — an officer must do everything by hand, hop by hop

Our differentiation: we're not trying to out-build Chainalysis. We're building the *free, self-hostable, India-context-first* slice of what they do — automated clustering + a confidence-scored "most likely exit point" report, tuned specifically to the workflow of a victim

complaint coming in through India's cybercrime portal, rather than a general-purpose enterprise compliance dashboard.

Advantages

- Built entirely on **real, public data** — Bitcoin and Ethereum ledgers are open, so the demo works on a live, real trace, not mocked data.
- Directly maps to a **named national workflow** (1930 helpline → wallet address → investigator), which is exactly the kind of concrete, government-usable story SIH judges reward.
- **Visually strong demo**: a force-directed graph lighting up hop by hop, ending in a flagged exchange, is far more compelling in a 5-minute slot than a dashboard of numbers.
- **Low expected competition** — the topic sounds intimidating, so most teams self-select into safer/more familiar ideas.
- Extensible: architecture generalizes to Ethereum, and later to other chains, without a redesign.

Disadvantages

- **Cross-chain laundering is a wall we can't fully solve in a hackathon.** Once funds move through a swap into Monero or a cross-chain bridge, on-chain tracing effectively breaks (this is true even for Chainalysis/TRM — they only maintain confidence through swaps probabilistically, not deterministically).
- **Attribution data is inherently incomplete.** Public exchange-address tag lists (WalletExplorer, GraphSense) don't cover every exchange, especially smaller or offshore ones — so some traces will dead-end honestly, which needs to be communicated to judges as a known limitation, not hidden.
- **Legal/ethical framing matters** — this must be pitched and built strictly as an investigator-assist tool for a specific, victim-initiated complaint, not a general surveillance/deanonymization engine, otherwise it invites the wrong kind of scrutiny.
- Less "flashy" to a lay judge than a computer-vision demo unless the visualization is designed carefully.

The Hard Part — and How We Get Past It

The hard part is the obfuscation-detection layer (step 4). Anyone can write a graph BFS in a day; distinguishing "this is a normal exchange consolidating deposits" from "this is a

scammer layering funds to break the trail” is a genuinely hard pattern-recognition problem, and it’s the one piece that makes this submission different from a weekend Etherscan script.

How to overcome it:

- Don’t try to build a state-of-the-art model in 36 hours. Start with **hand-coded heuristic rules** used in real forensic literature (rapid fan-out ratio, timing burst detection, round-number splitting, “peeling chain” detection) — these alone will beat a naive graph dump and are fast to implement and explain to judges.
 - Layer a **simple supervised classifier** (gradient-boosted trees on hand-engineered graph features, not a from-scratch GNN) trained on a small labeled set built from known historical scam/mixer cases that are publicly documented (e.g., past exchange-hack post-mortems where the laundering path was later published by researchers).
 - Keep the GNN/deep-learning version as a **“future work” slide**, not a live dependency — judges respect an honest, working heuristic system over a broken “AI” system that overpromises.
-

2. SIH26127 — City-Wide AI Engine for Multi-Camera ANPR Trajectory Tracking and Urban Traffic Analytics

Theme: Smart Automation · **Sponsoring Organisation:** Bharat Electronics Limited

Problem Statement

Cities already have ANPR cameras at junctions, toll points, and command-and-control feeds — but each camera reads plates in isolation. There is no system that stitches a single vehicle’s path *across* multiple, non-overlapping cameras into one continuous city-wide trajectory, which is what’s actually needed for real urban traffic analytics (route-level congestion, stolen-vehicle corridor tracking, ambulance green-corridor planning) rather than isolated point reads.

How We’re Going to Solve It

1. **Per-camera detection** — YOLO-class object detector for vehicles + an OCR stage for plate reads at each camera feed, exactly like existing ANPR does today.
2. **Re-identification layer (the actual ask in the title)** — for every detected vehicle, extract a compact appearance embedding (color, shape, silhouette) alongside the plate read, so the vehicle can still be matched across cameras even when the plate is blurred, angled, or unreadable in a given frame.

3. **Cross-camera graph stitching** — link detections across cameras using a scoring function that combines plate-match confidence, appearance-embedding similarity, and a travel-time prior (a car seen at Camera A at 10:00 can't plausibly appear at a camera 8 km away at 10:01 — physically implausible links get down-weighted).
4. **Trajectory construction** — chain the linked detections into a single, timestamped path per vehicle across the whole camera network — effectively a live “digital twin” of vehicle movement.
5. **Analytics layer on top** — corridor-level congestion heatmaps, anomaly flags (a vehicle looping the same block repeatedly, or entering a restricted zone), and a green-corridor path planner that can pre-clear signals along an ambulance's predicted route.

Current Solutions & How Ours Differs

Existing system	What it does	Gap
Existing city ANPR / Safe City / ICCC feeds	Reads plates at a single point (junction or toll gantry), flags known blacklisted plates against VAHAN	Each camera is a silo — no persistent, cross-camera trajectory is built; an officer has to manually correlate reads from different junctions
NHAI's ANPR-FASTag barrier-less tolling	Combines ANPR + FASTag RFID to identify vehicles at toll points without stopping	Purpose-built for toll billing, not general trajectory analytics; doesn't stitch a path across a city's full camera network
Recent MoRTH move to share FASTag/ANPR toll data with investigators	Lets agencies query specific vehicles at toll points for post-hoc investigation	Still point-based lookups (was the vehicle <i>here</i> at <i>this</i> toll plaza), not a continuous, real-time trajectory or predictive analytics layer
Academic/industrial multi-camera vehicle re-ID research (e.g., AI City Challenge benchmarks)	Solves the re-ID problem in a research setting on benchmark datasets	Not deployed as an end-to-end city analytics product; no traffic-planning layer on top

Our differentiation: every deployed system in India today is a **point sensor**. Nobody has closed the loop into a genuine city-wide trajectory graph with an analytics layer on top — that's precisely the part of the PS title (“Trajectory Tracking,” not just “ANPR”) most teams will skip because it's the hardest part.

Advantages

- A **real public benchmark dataset exists** for exactly this problem (multi-camera vehicle re-identification data from research challenges), so the model can be trained and demoed on legitimate data instead of fabricated footage.
- **Very strong visual demo** — a live map with vehicle trails moving between camera icons is intuitive for any judge, technical or not, to grasp in seconds.
- Directly useful, concrete government pain point (traffic planning, stolen-vehicle corridors, emergency green corridors) with obvious ROI framing.
- Modular: even a partial build (say, working re-ID + trajectory stitching without the full analytics layer) is still an impressive, demoable core.

Disadvantages

- **Compute-heavy.** Running detection + OCR + embedding extraction across multiple simultaneous video streams in real time needs real GPU throughput — hard to guarantee smoothly on a laptop demo rig.
- **Privacy/surveillance optics** — a system that tracks every vehicle's city-wide movement needs a clear, stated purpose-limitation (traffic + law-enforcement corridors only) in the pitch, or it can read as mass surveillance to a cautious judge.
- Camera calibration/geometry (knowing the real-world distance and travel-time between two specific camera locations) is fiddly to fake convincingly without real city camera-network data.
- The "digital twin" framing is popular right now — even though *this specific* multi-camera stitching angle is under-attempted, some teams will pitch adjacent "smart traffic AI" ideas, so the demo needs to clearly show the cross-camera linkage, not just single-camera ANPR, or it'll blur together with generic traffic-AI pitches in judges' minds.

The Hard Part — and How We Get Past It

The hard part is step 3 — cross-camera linking without ground-truth camera geometry. In a real deployment you'd calibrate exact GPS coordinates and drive-time between every camera pair; in a hackathon you won't have that data for real city cameras.

How to overcome it:

- Use the **public benchmark dataset** (which does include camera geometry and ground-truth trajectories) to prove the re-ID + stitching algorithm actually works, quantitatively — this gives you a real accuracy number to put on a slide, which is far more convincing than a live demo alone.

- For the “city” demo layer, **simulate a small synthetic camera network** (e.g., 4–6 virtual cameras placed on a real map of your own city, with estimated travel times) and feed it recorded traffic footage — be transparent that the network topology is illustrative while the underlying re-ID engine is real and benchmarked.
 - Scope the live demo to **2–3 cameras**, not a full city, so the compute stays manageable on a laptop/single GPU, and clearly state the scaling path (edge inference per camera + a lightweight central matching service) as the deployment architecture.
-

3. SIH26165 — AI/NLP Engine to Detect Serious Injury & Fatality (SIF) Precursors in Unsafe-Act/Unsafe-Condition and Near-Miss Reports

Theme: Smart Automation · **Sponsoring Organisation:** Oil India Limited

Problem Statement

Oil & gas sites generate thousands of near-miss and unsafe-act/unsafe-condition text reports. Most of these are logged, categorized, and filed away — but safety science (Heinrich’s and Bird’s accident pyramids) shows that serious injuries and fatalities are almost always preceded by a pattern of “minor” near-misses that share common precursors. The PS asks for an NLP engine that can read these free-text reports and surface which ones are actual **leading indicators** of a future serious incident, rather than just tagging them by category after the fact.

How We’re Going to Solve It

1. **Ingestion & normalization** — parse free-text near-miss/unsafe-act reports (field-worker-written, often short and informal), normalize terminology (industry jargon, abbreviations, site-specific phrasing).
2. **Precursor feature extraction** — use NLP (embeddings + rule-based entity extraction) to pull out structured signals from the free text: hazard type, equipment involved, location/process area, human-factor cues (fatigue, rushed procedure, missing PPE), and severity-of-potential language.
3. **Causal/co-occurrence graph mining** — instead of classifying each report in isolation, mine the *history* of reports for the same equipment/location/hazard-type combination and build a co-occurrence graph — clusters of “boring” reports sharing the same root cause are exactly what safety pyramid theory says precede a serious event.

4. **Leading-indicator scoring (our novel layer)** — score each new incoming report not just by its own severity language, but by how closely its precursor pattern matches historical clusters that are documented to have escalated, producing a proactive “watch this” flag rather than a static category label.
5. **Explainable dashboard** — for EHS officers: a ranked watchlist of high-risk precursor clusters, with the actual report snippets and shared root-cause tags that justify the ranking (explainability matters enormously here — an EHS officer won’t act on a black-box score).

Current Solutions & How Ours Differs

Existing solution	What it does	Gap
EHS software suites (Enablon, Intellex, SafetyStratus, IBM Maximo Safety-type platforms)	Digitize near-miss logging, basic categorization/tagging, dashboards and compliance reporting	Reactive — they record and categorize what happened; they don’t mine cross-report precursor patterns to flag what’s <i>about</i> to happen
Academic NLP-on-safety-reports research (text mining for accident cause classification, topic modeling on incident narratives)	Demonstrated in refineries, coal mines, and construction — good accuracy classifying reports by cause/severity after the fact	Published research prototypes, not integrated, real-time products; almost all are <i>post-hoc classification</i> , not <i>forward-looking leading-indicator scoring</i>
Emerging LLM-based near-miss triage pilots (a few oil & gas majors have piloted LLMs to auto-categorize and route incoming reports)	Faster categorization, better tagging, improved reporting-rate	Still fundamentally a smarter filing cabinet — categorize-and-route, not connect-the-dots-across-reports-to-predict-escalation

Our differentiation: everything that exists today either **logs** near-misses or **classifies** them individually. Nobody in the deployed-product space is explicitly built around Heinrich’s/Bird’s pyramid logic — cross-referencing many “minor” reports to flag the precursor *pattern* itself as the risk signal, before any serious incident occurs.

Advantages

- **Genuinely low competition** — this PS sounds dry (“NLP on text reports”), so most teams chasing a flashy computer-vision or chatbot demo will skip it entirely.
- **Real public data exists as a stand-in** — occupational safety narrative datasets (e.g.,

publicly available accident/near-miss text corpora from safety regulators) can be used to prototype and validate the pipeline even without OIL's proprietary data.

- **Strong, quantifiable ROI story** for a PSU judge panel: "X% of serious incidents in the historical record were preceded by a detectable precursor cluster we could have flagged N days in advance" is a compelling, numbers-backed pitch.
- Explainability-first design plays well with how seriously safety-critical industries take auditability — an "explainable AI" angle is a genuine differentiator, not just a buzzword, here.

Disadvantages

- **Not visually flashy.** No live video, no graph animation by default — the demo lives or dies on how well the dashboard and the "here's a real incident we would have caught early" narrative are presented.
- **Real proprietary OIL data won't be available** for a student team, so results have to be built and validated on public proxy datasets, then framed honestly as "validated methodology, ready to be re-trained on OIL's actual historical reports."
- **Precursor labeling is inherently noisy** — near-miss reports are informally written, and "this pattern preceded a serious incident" ground truth is rare and hard to construct without real historical linkage data.
- Domain vocabulary (industry-specific jargon, abbreviations, site codes) needs at least some domain research to get right, or the NLP layer will misparse a lot of real-world text.

The Hard Part — and How We Get Past It

The hard part is step 3/4 — proving the leading-indicator signal is real, not just noise, without OIL's actual historical escalation data (i.e., which near-miss clusters actually preceded a real serious incident).

How to overcome it:

- Don't fabricate a labeled "this precursor pattern definitely predicts serious incidents" dataset — that would be dishonest and judges in this domain can smell it. Instead, **build the pipeline generically** (precursor extraction → clustering → co-occurrence scoring) and validate it on a **public dataset that does have documented escalation outcomes** (there is published safety-science literature, e.g., studies on coal-mine or construction accident report corpora, where researchers have already linked minor-report clusters to major outcomes) — cite that validation, then present the OIL-specific version as "same engine, ready to be pointed at OIL's proprietary corpus."

- Frame the demo around **one well-documented historical case study** (a real, publicly written-up industrial incident with a known precursor trail) and show the system correctly flagging the precursor cluster in hindsight — a single, concrete, honest “we would have caught this” story is more convincing than a large, hand-wavy dataset.
- Keep the scoring model **simple and rule-augmented** (co-occurrence counts + a small supervised layer) rather than a large opaque model — in a safety-critical explainability-first pitch, a system judges can actually interrogate line-by-line will land better than a bigger black box.

Quick Comparison at a Glance

Criterion	SIH26183 (Crypto Trail)	SIH26127 (Multi-Cam ANPR)	SIH26165 (Safety NLP)
Real data available	Yes — live public blockchain data	Yes — public re-ID benchmark dataset	Partial — public proxy datasets only
Expected internal-round competition	Very low	Moderate (traffic-AI is a popular space, but this exact angle is rare)	Very low
Demo visual impact	High (live fund-flow graph)	Very high (live map + trails)	Low-moderate (dashboard-driven)
Compute demands	Low-moderate	High (multi-stream video)	Low
Hardest unsolved piece	Obfuscation/mixing detection	Cross-camera linking without real geometry	Proving the leading-indicator signal is real
Judge-narrative strength	Strong (named citizen helpline workflow)	Strong (concrete city pain point)	Strong but needs careful, numbers-led pitching